

EmoRecAgent: Privacy-Preserving Multi-Agent Sequential Recommendation for E-Commerce via Sentiment-Aware Profiling and Guarded Reranking

EmoRecAgent Project

Manuscript prepared for submission; affiliation and author list withheld pending internal review.

Abstract—Recommender systems increasingly rely on large language models (LLMs) to interpret customer intent beyond what collaborative signals alone can express, yet most LLM-augmented designs either ignore free-text reviews when scoring the full catalog, or send those reviews to third-party APIs — a data-governance liability many e-commerce operators cannot accept — and unconstrained LLM agents remain prone to hallucinated preferences or unstable rankings. We present EmoRecAgent, a privacy-preserving multi-agent framework whose central idea is that LLM agents should not decide in isolation: they perform the main language work, but under explicit algorithmic constraints and computed evidence. Five cooperating agents share the recommendation. The Sequential Ranking Agent — a deep learning sequential encoder, not an LLM — owns full-catalog scoring from purchase timestamps. The Aspect-Sentiment Extraction Agent uses a local LLM as the primary extractor of aspect–opinion–polarity triples from each review; a supervised PyABSA model is kept only as a justified fallback when that LLM call fails, times out, or returns empty or unusable output. A Dynamic User-Profiling Agent then aggregates the kept triples by a closed-form time-decay rule into an aspect-salience vector $w_u(a, t)$, and a Preference-Manifesto Agent verbalizes those weights into a short statement T_u . Finally, the Guarded Reranking Agent aligns T_u into the sequential embedding space with an InfoNCE-trained MLP, lets a local LLM propose a listwise reorder of a bounded top- K pool, and blends that proposal with the sequential prior under a learned soft trust $\alpha_u = g_\psi(\mathbf{z}_u)$ conditioned on alignment and displacement features. All LLM calls run on open-weight models (Qwen2.5-3B/7B-Instruct) behind a self-hosted endpoint, so customer data never leaves the operator’s infrastructure. On Beauty & Personal Care under full-catalog ranking, the constrained multi-agent system improves HR@10 by 11.0% over the same sequential backbone alone (0.1047→0.1162); gains are larger on Sports & Outdoors, near-neutral on Toys & Games, and mixed on Yelp, and a Yelp ablation shows that removing the guardrail or the LLM call is inferior to the guarded configuration. We evaluate on five benchmarks spanning Amazon Reviews 2023 and Yelp (including a literature-anchored Yelp_AC reproduction) and position EmoRecAgent as constrained multi-agent cooperation under an operator boundary, complementary to open-ended LLM-agent planners and multi-objective LLM rerankers.

models, multi-agent systems, sequential recommendation, listwise reranking, aspect-based sentiment analysis, self-hosted inference, e-commerce personalization.

I. INTRODUCTION

Online retail and local-service platforms depend on recommendation to compress an enormous catalog into a short, relevant list for each visitor. Classical collaborative filtering and sequential models are efficient and scale well, but they see a user only as a stream of item identifiers and timestamps; they cannot read the sentence a customer wrote about why a product disappointed them, and they cannot distinguish “I buy sunscreen every summer” from “I finally found a sunscreen that does not sting my eyes.” Large language models (LLMs) close part of this gap: they read free text, infer intent, and can explain a recommendation in the same language the customer used. Recent surveys count this shift as one of the fastest-growing directions in the field, with LLMs used as feature encoders, scoring functions, or fully autonomous recommendation agents [1].

That shift, however, has produced a form of recommendation that most retailers cannot deploy as-is. The dominant recipe — send the user’s purchase history and review text to a hosted LLM API and read back a ranked list or an explanation — requires exporting exactly the data that data-protection regulation, corporate data-governance policy, and customer trust treat as sensitive. This is not a hypothetical concern raised only by this paper: dedicated surveys document memorization and reconstruction attacks against LLM-based recommenders [2], membership-inference attacks that can determine, from a model’s behavior alone, whether a specific customer’s interaction history appeared in a prompt [3], and privacy-utility trade-off studies that had to design obfuscation and de-obfuscation pipelines specifically because sending full purchase histories to a remote LLM leaks which sensitive products a customer bought [4]. On the deployment side, the same literature repeatedly names inference latency, serving cost, and hallucination as open, largely unsolved obstacles to using LLMs directly inside a live recommendation loop [1]. Two problems therefore sit unresolved

Index Terms—recommender systems, large language

at the center of LLM-based recommendation: (i) how to obtain LLM-quality personalization without exporting customer data to a third party, and (ii) whether a model an operator can actually afford to self-host — typically an order of magnitude smaller than the frontier API models used in most published LLM-recommender studies — gives up meaningful quality to buy that independence back. Existing agentic and LLM-rerank recommenders [5], [6], [7], [8] are evaluated almost exclusively against commercial APIs, so the second question is, to our knowledge, rarely measured directly on a shared recommendation benchmark.

EmoRecAgent is built around that pair of requirements, but with a sharper design claim than “add an LLM to the stack.” The language agents do the main interpretive work — reading reviews, drafting a preference statement, proposing a shortlist reorder — yet they do not act alone. Each generative step is wrapped by a definite algorithm or constraint: a deep sequential ranker that owns full-catalog scores, a closed-form time-decay profile, a latent alignment of T_u into that sequential space, a bounded candidate pool, and a learned soft trust weight that blends an LLM shortlist proposal with the sequential prior. In that sense the breakthrough is not a larger prompt, but a disciplined division of labor in which the LLM reasons under rules that were computed before the call. Concretely, the Sequential Ranking Agent — a dedicated neural model — scores the catalog from identifiers and timestamps alone; the Aspect-Sentiment Extraction Agent uses a local LLM as the primary ABSA reasoner, with PyABSA retained only for failure cases that would otherwise leave the pipeline empty; profiling and the manifesto turn the kept triples into T_u ; and the Guarded Reranking Agent synthesizes these signals before accepting any shortlist permutation. Every LLM call runs on an open-weight model behind a self-hosted, OpenAI-compatible endpoint, so no review, purchase record, or derived preference statement leaves the operator’s infrastructure: an *operator-boundary* constraint, not a cryptographic privacy mechanism. We also treat local-model utility as something to measure rather than assume, and report, domain by domain, how much the language agents add over the neural backbone alone. Figure 1 sketches this idea; the concrete agent architecture appears in Figure 2.

We summarize the contribution of this paper as:

- **Constrained LLM agents rather than isolated generation.** The core idea of EmoRecAgent is that language agents remain the primary reasoners over free text, yet every generative decision is bound to an algorithm or hard constraint (Sections III–III-G). The Sequential Ranking Agent is a deep time-interval encoder — not an LLM — that supplies $\pi_u^{(1)}$ (Section III-B). The Aspect-Sentiment Extraction Agent uses a local LLM as the main ABSA worker, with PyABSA only as a fallback when that call fails. Profiling fixes $w_u(a, t)$ by a closed-form time-decay rule; the manifesto verbalizes that vector rather than inventing

aspects freely; and the Guarded Reranking Agent first computes latent alignment and pool evidence, feeds those results into a local LLM for the listwise decision, and blends that decision with $\pi_u^{(1)}$ under a *learned soft trust* $\alpha_u = g_\psi(\mathbf{z}_u)$ (Section III-F). This complements multi-objective LLM rerankers such as LLM4Rerank [7] and open-ended multi-agent planners such as MACRec [8].

- **An operator-boundary serving stack with fail-closed language agents.** All LLM calls use locally hosted Qwen2.5-3B/7B-Instruct models through a self-hosted endpoint. When the primary ABSA LLM fails, the agent may keep a PyABSA fallback; when rerank parsing fails, the agent sets soft trust to zero and keeps the sequential (or cross-user-boosted) order (Sections III-C, III-F, III-G). We frame this as a deployment constraint that many retailers already require, not as a new differential-privacy or federated-learning method.
- **A multi-domain empirical study of when the language agents help.** On three Amazon Reviews 2023 categories and two Yelp tracks — including a Yelp_AC reproduction of a published TiSAS-Rec preprocessing recipe [9] — we report the lift of the full multi-agent system over the *same* sequential backbone, together with a guardrail/LLM ablation. The language agents help on Beauty and Sports, are near-neutral on Toys, and mixed on Yelp; we treat this domain dependence as a finding, not a footnote (Sections IV–V).

The remainder of the paper is organized as follows. Section II reviews sequential and graph-based recommenders, LLM-agent recommenders, and privacy concerns in LLM-based recommendation. Section III presents the EmoRecAgent architecture and its mathematical formulation. Section IV describes the datasets, baselines, and evaluation protocol. Section V reports and discusses results. Section VI concludes with limitations and future work.

II. RELATED WORK

A. Time-Aware and Sequential Recommendation

Sequential recommenders model a user’s next action as a function of their ordered interaction history. SASRec [10] first showed that a purely self-attentive encoder over item order, with no recurrence or convolution, matches or exceeds Markov-chain and RNN-based models while training an order of magnitude faster, establishing self-attention as the default building block for next-item prediction. BERT4Rec [11] replaced SASRec’s left-to-right view with a bidirectional encoder trained via a Cloze (masked-item) objective, so each item conditions on both earlier and later context. TiSASRec [12] extends the self-attentive line orthogonally by injecting absolute-position and pairwise time-interval embeddings into attention, so that identical item sequences at different real-world time gaps yield different next-item predictions. EmoRecAgent’s Sequential Ranking Agent builds on that time-interval

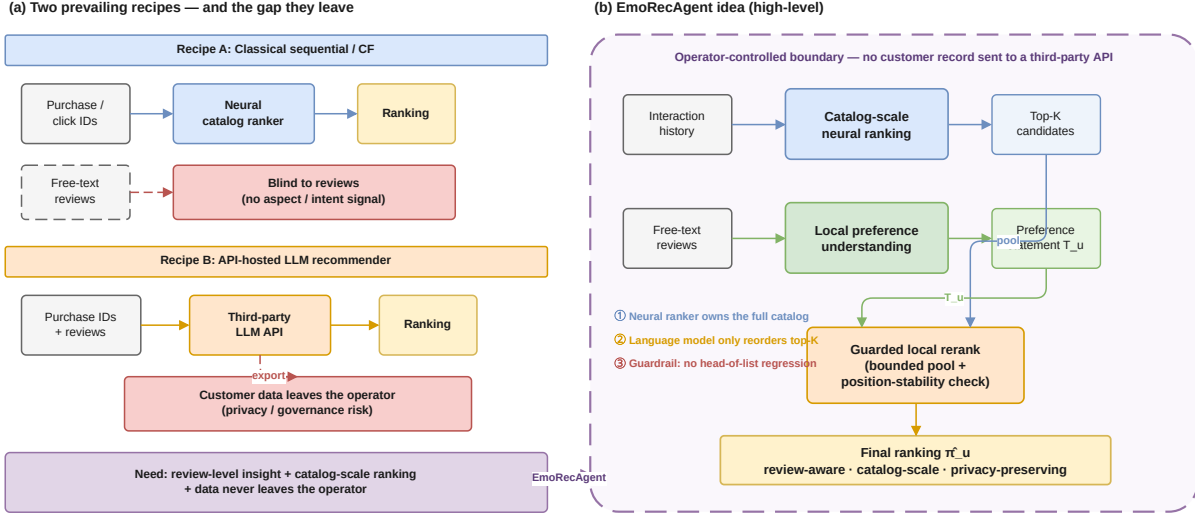


Fig. 1. High-level idea of EmoRecAgent (not the module architecture). (a) Two prevailing recipes and the gap they leave: classical sequential/CF models scale over the catalog but are blind to free-text reviews, while API-hosted LLM recommenders exploit review language at the cost of exporting customer data and unconstrained generation. (b) EmoRecAgent keeps both strengths inside an operator-controlled boundary: LLM agents do the main language work, but decide under algorithmic constraints — a neural full-catalog ranker, a formula-defined profile, latent alignment of T_u , and a learned soft trust blend of any local listwise reorder with the sequential prior.

insight rather than on order-only models such as SASRec or BERT4Rec, then uses dense full-catalog CE training and emits an explicit full-catalog prior $\pi_u^{(1)}$ that later language agents may only softly blend under a learned trust weight (Section III-B). The guarded-rerank stage therefore treats time-aware sequential scores as first-class computed evidence, distinct from the LLM agents that follow. A parallel line of work argues that transformer-based sequential recommenders systematically mis-allocate attention to noisy or weakly relevant history positions, and proposes attention-calibration modules attachable to TiSASRec-style backbones [9]; we reuse this paper’s own Yelp pre-processing recipe (post-2019 transactions, 5-core filtering, leave-one-out evaluation) for one of our five benchmarks so that our sequential-ranking numbers are directly checkable against an externally reported result.

B. Graph-Based and Contrastive Collaborative Filtering

Graph convolutional recommenders treat the user-item interaction record as a bipartite graph and propagate embeddings along edges. LightGCN [13] showed that removing feature transformation and nonlinear activation from graph convolution — keeping only normalized neighborhood aggregation — produces a simpler and often stronger collaborative-filtering signal than heavier graph neural networks. SimGCL [14] and its extension XSimGCL [15] show that most of the benefit attributed to graph-augmentation-based contrastive learning can be reproduced, at a fraction of the computational cost, by adding uniform random noise directly to the embedding space, which motivates using them as strong, inexpensive contrastive baselines. CORE [16] instead targets the

representation-space mismatch between a session encoder and the item embedding table, aligning both into a single consistent space for session-based next-item prediction. All four appear in our baseline suite as representative non-LLM neural recommenders that ignore review text entirely.

C. Review-Aware and Sentiment-Aware Recommendation

A separate, longer-standing line of work brings review text directly into the rating or ranking objective rather than treating it only as raw material for an LLM prompt. DeepCoNN [17] pioneered this with two coupled convolutional networks — one over reviews a user wrote, one over reviews an item received — joined by a factorization-machine layer, showing review text alone can substantially outperform rating-only matrix factorization in the sparse regime that also motivates our own use of reviews. NARRE [18] added an attention weight over each review to identify which are actually informative, using the same weights to both improve rating prediction and surface a review-level explanation. Neither model extracts a structured, aspect-level signal, however: both treat a review as one undifferentiated span. Aspect-based sentiment analysis (ABSA) closes that gap by decomposing a review into (aspect, opinion, sentiment) triples, a formulation traced to the opinion-mining framework of Hu and Liu [19]; recent work such as ASKAT [20] fuses these triples into a knowledge-graph attention network so a recommender can distinguish an aspect a user likes from one the same user explicitly dislikes, rather than collapsing a review into one polarity score. EmoRecAgent’s Aspect-Sentiment Extraction Agent (Section III-C) follows this structured

ABSA tradition, but puts a local LLM in the primary role of reading each review and emitting triples; a supervised PyABSA model is retained only as a fallback when that generative step fails. The kept triples are then aggregated by a closed-form time-decay rule into $w_u(a, t)$ and verbalized, rather than propagated through a knowledge graph.

D. LLM Agents That Rank, Rerank, or Simulate Users

Before agentic designs became common, a first wave of work asked whether an LLM itself, rather than an LLM controlling a classical model, could serve directly as the recommender. P5 [21] reformulated five recommendation tasks (rating, sequential, explanation, summarization, direct recommendation) as one text-to-text objective over a shared encoder-decoder model driven by personalized prompts, showing one checkpoint generalizes zero-shot across templates. TALLRec [22] instead lightly fine-tunes an open-weight LLM (LLaMA-7B) with low-rank adaptation on a few dozen labeled examples and reports it clearly outperforms in-context prompting of a much larger closed model; we read this, alongside our own results in Section V, as literature-consistent evidence that a small, locally hostable model need not be at a quality disadvantage relative to a frontier API model on recommendation-shaped tasks.

Building on this LLM-as-recommender foundation, a second family of work puts an LLM either in control of, or in dialogue with, a classical recommender. InteRecAgent [5] treats the LLM as a conversational controller that calls classical retrieval and ranking models as tools, coordinating a shared candidate pool and a persistent user-profile memory. RecMind [6] instead gives the LLM its own planning and memory loop, using a “self-inspiring” strategy that reuses previously explored reasoning states, and reports results competitive with fine-tuned expert recommenders on several recommendation-style tasks without any parameter update. LLM4Rerank [7] is architecturally closest to our Guarded Reranking Agent: it casts multiple reranking objectives (accuracy, diversity, fairness) as nodes in a graph that an LLM traverses with chain-of-thought reasoning, guided by a customizable goal. EmoRecAgent takes a complementary specialization: LLM agents still do the main language work, but each decision is bound to an algorithm or constraint — a manifesto grounded in formula-defined salience weights, a bounded candidate pool, InfoNCE alignment of T_u , and a learned soft trust blend — rather than a free multi-objective graph walk. MACRec [8] deploys a society of specialized agents (manager, analyst, reflector, searcher) that collaborate on recommendation tasks and is the closest published multi-agent analogue to our division of labor. EmoRecAgent shares that cooperative factorization, but insists that agents do not decide in isolation: the LLM remains the primary ABSA and rerank reasoner, PyABSA covers only ABSA failure cases, w_u stays the closed-form vector in Eq. (10), and any listwise reorder is softly blended with the deep Sequential Ranking Agent under a learned trust weight over alignment and displacement features.

A related but distinct use of LLM agents is to *simulate* users rather than to serve as the recommender. Agent4Rec [23] instantiates roughly a thousand generative agents with profile, memory, and emotion-driven reflection, and uses them to study how existing recommenders behave under more human-like, simulated feedback; we include it as an agentic baseline in our experiments because, when repurposed to directly rank items in our benchmarks, it lets us contrast a general-purpose LLM-agent simulator against a system purpose-built for ranking. AgentCF [24] pushes the idea further by modeling both users *and* items as interacting language agents that reflect against real logs to propagate collaborative signal purely in language space. ChainRec [25] trains a planner to select and order tool calls adaptively per user, reporting its strongest gains on cold-start and evolving-interest slices of AgentRecBench [26]. We view ChainRec and AgentRecBench as complementary interactive protocols; EmoRecAgent instead studies grounded cooperation under the static, full-catalog Protocol B in Section IV, where the scientific question is not open-ended tool routing but whether DL- and formula-anchored agents reduce the failure modes of unconstrained LLM reranking.

More broadly, MACRec and ChainRec descend from general-purpose multi-agent LLM frameworks developed outside recommendation. Park et al. [27] showed a persistent memory stream, retrieval, and periodic reflection suffice for believable agent behavior — the pattern Agent4Rec and AgentCF borrow to simulate users. AutoGen [28] further generalizes conversable agents into flexible topologies. EmoRecAgent participates in this multi-agent lineage, but with a different emphasis: the LLM agents are the primary workers on free text, yet they decide under constraints and algorithms rather than in isolation. They interpret reviews, draft T_u , and propose a shortlist permutation; when a generative step fails, the system can fall back to a supervised ABSA extractor, a formula-defined salience vector, or the sequential ranking already scored by the deep Sequential Ranking Agent (Eqs. (9)–(18)). Cooperation therefore does not mean unconstrained generation over vague evidence.

E. Privacy and Local Deployment of LLMs for Recommendation

The privacy dimension of LLM-based recommendation has received comparatively less attention than accuracy, but a growing body of work makes the risk concrete. Luo et al. [2] survey privacy attacks across the LLM-recommendation life cycle and show that even providers’ assurances do not prevent a model from inferring user attributes and preferences directly from interaction history supplied in a prompt. He et al. [3] design four membership-inference attacks against in-context-learning LLM recommenders and confirm that at least two — direct inquiry and a poisoning attack — succeed with practically meaningful advantage, meaning an adversary can determine whether a specific user’s history was used

to build a recommendation prompt. Khezresmaeilzadeh et al. [4] take the risk as given and design an obfuscate-then-de-obfuscate pipeline so that a server-side LLM never sees a user’s full, unmasked purchase history; notably, they report that their privacy-preserving pipeline is efficient enough to run on consumer-grade hardware, which is consistent with our own design choice to self-host rather than obfuscate: if the model is already local, there is no channel to obfuscate against. Broad surveys of LLM-enhanced recommender systems repeatedly list inference latency, serving cost, and privacy alongside each other as the field’s open deployment obstacles [1].

Outside the LLM literature specifically, federated learning offers a longer-standing answer to the same data-governance question. Ammad-ud-din et al. [29] introduced a federated collaborative filter in which each client computes its own gradient update locally and only the aggregated item-factor update reaches the server, so raw interaction logs never leave the device, and show this matches centralized matrix factorization in accuracy. This confirms privacy-by-architecture is achievable for classical CF, but does not by itself extend to the free-text understanding EmoRecAgent’s LLM stage requires: federating a matrix-factorization update is a different problem from self-hosting an LLM call that reads a paragraph of natural language. EmoRecAgent’s response is likewise architectural rather than cryptographic or federated: keep the LLM off the request path for full-catalog scoring, bound its job to a small already-narrowed candidate pool, and run it on infrastructure the operator controls end-to-end — an operator-boundary constraint that removes the remote-prompt channel those attacks exploit, without claiming a formal privacy proof.

F. Positioning

EmoRecAgent’s claim is a multi-agent design for e-commerce ranking in which LLM agents do the main language work, but never in isolation: each generative decision is constrained by an algorithm or by evidence that was already computed. Relative to LLM4Rerank, the path is review $\rightarrow$ (LLM-primary ABSA, PyABSA only on failure) $\rightarrow$ formula $w_u \rightarrow T_u \rightarrow$ (InfoNCE alignment + LLM reorder + soft trust blend), rather than multi-objective CoT graph search. Relative to MACRec-style agent societies, the distinguishing idea is constrained decision-making: the LLM remains first-class, while PyABSA, the closed-form profile, latent alignment of T_u , and a learned soft trust over alignment/displacement features bound how strongly that LLM may move the list when language is empty, malformed, or too weakly supported. Relative to privacy-oriented LLM-rec work, we contribute an operator-boundary deployment pattern (local models + fail-closed fallbacks), not a cryptographic protocol; a controlled local-versus-API measurement on the same prompts remains future work (Section V-F).

III. THE EMORECAGENT FRAMEWORK

A. Problem Formulation and Notation

Let $\mathcal{U}$ and $\mathcal{I}$ be the sets of users and items. Each interaction is a tuple (u, i, t, r) with user $u \in \mathcal{U}$, item $i \in \mathcal{I}$, Unix timestamp t , and optional free-text review r (we write $r = \emptyset$ when none is present). For a query time t_q , the history available to the system is the time-ordered sequence of earlier interactions

$$\mathcal{H}_u(t_q) = ((i_k, t_k, r_k))_{k=1}^{n_u(t_q)}, \quad t_1 < \dots < t_{n_u(t_q)} < t_q, \quad (1)$$

where $n_u(t_q)$ is the length of that history. We also write $\mathcal{H}_u^{\text{id}}(t_q) = ((i_k, t_k))_k$ for the identifier–time view used by the sequential ranker, which never reads review text.

The goal is to rank the catalog so that the held-out next item i_u^* sits as high as possible. EmoRecAgent approaches this with five cooperating agents whose design principle is easy to state: LLM agents perform the main language work, but they decide under constraints and algorithms rather than in isolation. The Sequential Ranking Agent first builds a full-catalog ranking $\pi_u^{(1)}$ from $\mathcal{H}_u^{\text{id}}(t_q)$ alone. In parallel, the Aspect-Sentiment, Profiling, and Manifesto agents turn reviews into a short preference statement T_u , with the LLM reading free text and the closed-form profile fixing which aspects matter. The Guarded Reranking Agent then combines these signals into the final ranking

$$\hat{\pi}_u = \mathcal{G}(\pi_u^{(1)}, T_u, \text{co}, \mathcal{H}_u(t_q)), \quad (2)$$

where $\mathcal{G}$ is the guarded procedure in Section III-F and co is a training-set co-occurrence table defined there. Throughout, $\pi(j)$ denotes the item at position j of a ranking π , and $\text{rank}(i, \pi)$ its 1-indexed position.

Figure 2 summarizes the five agents. Agent ① (Section III-B) is a deep learning sequential encoder that scores the full catalog from identifiers and timestamps alone — it does not call an LLM, and later language steps must respect the ranking it produces. Agent ② (Section III-C) uses a local LLM as the primary aspect extractor; PyABSA is only a fallback when that LLM fails. Agent ③ (Section III-D) aggregates the kept triples into salience weights $w_u(\cdot, t)$ by a fixed time-decay rule, not by free-form generation. Agent ④ (Section III-E) verbalizes those weights into T_u . Agent ⑤ (Section III-F) aligns T_u via an InfoNCE-trained MLP, lets the LLM propose a shortlist order from a computed evidence pack, and softly blends that proposal with $\pi_u^{(1)}$ under $\alpha_u = g_\psi(\mathbf{z}_u)$ (Eqs. (17)–(18)). Only Agents ②–④ and the listwise step of Agent ⑤ call the self-hosted LLM layer (Section III-G).

B. Sequential Ranking Agent

The Sequential Ranking Agent is a *deep learning* sequential encoder — not a language-model agent. It never reads review text, never issues an LLM call, and produces a full-catalog ranking $\pi_u^{(1)}$ from item identifiers and timestamps alone. Architecturally we follow the TiSASRec lineage [12]: pairwise time intervals enter self-attention so that purchase *pacing*, not only item order, shapes the user

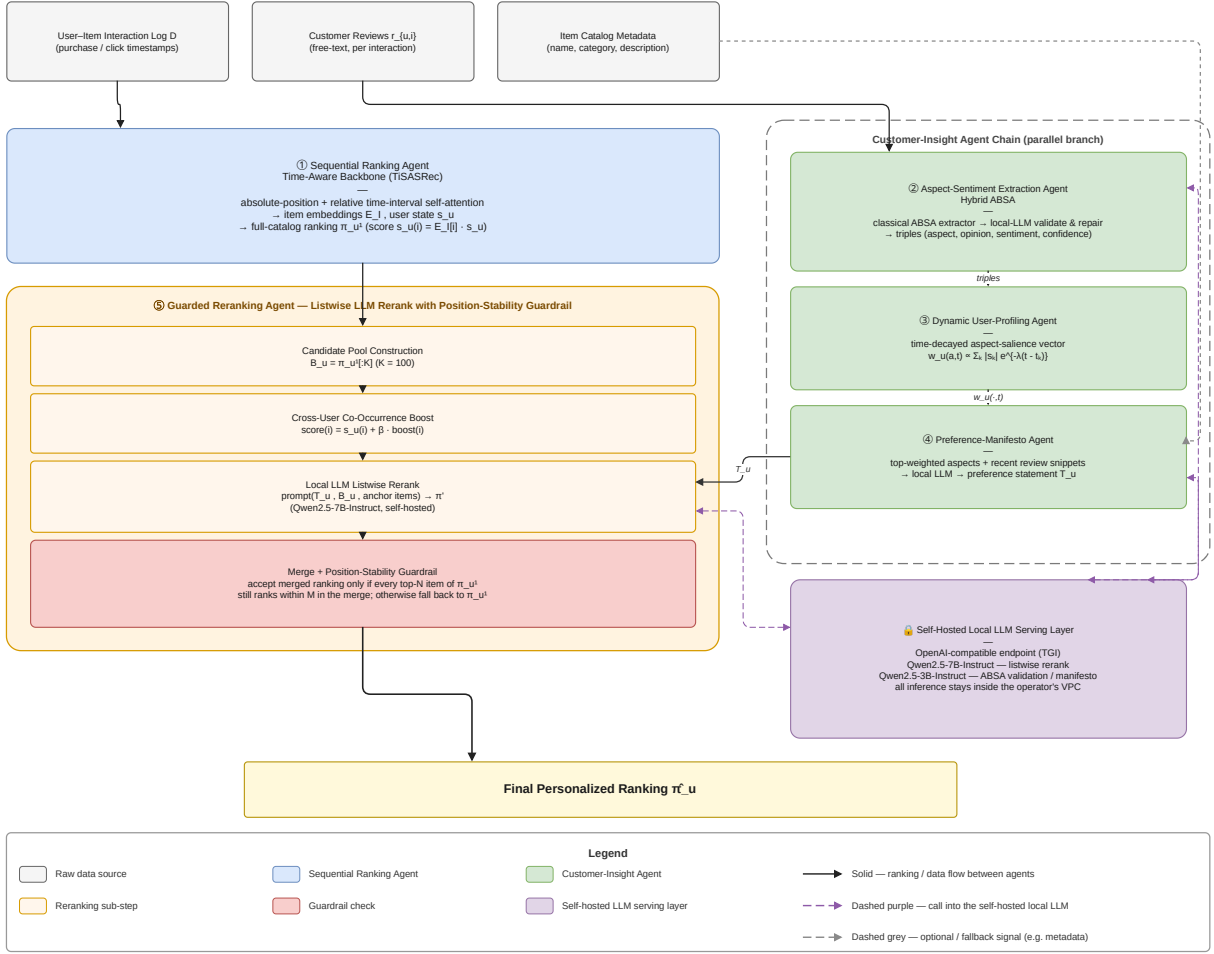


Fig. 2. EmoRecAgent as five cooperating agents (①–⑤). Solid arrows carry ranking scores and other computed signals; dashed grey marks the optional metadata fallback; dashed purple marks calls into the self-hosted local LLM layer. LLM agents do the main language work, but operate under algorithmic constraints rather than in isolation.

state s_u . We treat this encoder as a strong, reproducible numeric backbone rather than as the paper’s technical novelty; the contribution we emphasize is how later agents may *softly* trust an LLM reorder of $\pi_u^{(1)}$ (Section III-F). The external TiSASRec (BCE) checkpoint in Section IV shares that lineage but is trained under a different objective and recipe.

a) Input, time-interval attention, and scoring.:

From $\mathcal{H}_u^{\text{id}}(t_q)$ the agent keeps the most recent L interactions $(i_1, \dots, i_L)$ with timestamps $(t_1, \dots, t_L)$, left-padding shorter histories ($i_k = 0$ marks padding). Item, absolute-position, and pairwise time-interval embeddings follow TiSASRec: gaps are bucketed as

$$\Delta_{kl} = \min(\lfloor |t_k - t_l| / \tau_{\text{unit}} \rfloor, T_{\text{span}}), \quad (3)$$

and enter both the attention logits and the value stream through separate key/value interval tables $\mathbf{r}_{kl}^K, \mathbf{r}_{kl}^V$. With content projections $\mathbf{q}_k, \mathbf{k}_l, \mathbf{v}_l$ and absolute-position tables

$\mathbf{p}^K, \mathbf{p}^V$, causal attention at position k is

$$\begin{aligned} e_{kl} &= \frac{1}{\sqrt{d}} (\mathbf{q}_k^\top \mathbf{k}_l + \mathbf{q}_k^\top \mathbf{p}_l^K + (\mathbf{r}_{kl}^K)^\top \mathbf{q}_k), \\ \alpha_{kl} &= \frac{\exp(e_{kl})}{\sum_{l' \leq k} \exp(e_{kl'})}, \\ \mathbf{o}_k &= \sum_{l \leq k} \alpha_{kl} (\mathbf{v}_l + \mathbf{p}_l^V + \mathbf{r}_{kl}^V). \end{aligned} \quad (4)$$

Stacking B blocks and reading the last non-padding state yields $\mathbf{s}_u = \mathbf{h}_L \in \mathbb{R}^d$. Full-catalog scores and the prior ranking are

$$\begin{aligned} s_u(i) &= \mathbf{e}_i^\top \mathbf{s}_u, \\ \pi_u^{(1)} &= \text{argsort}_{i \in \mathcal{I}}^\downarrow s_u(i). \end{aligned} \quad (5)$$

b) Training recipe.: To match Protocol B inference we train with dense full-catalog cross-entropy rather than TiSASRec’s original sampled BCE. At every non-padding timestep k , logits $\ell_{u,k}(i) = \mathbf{h}_{u,k}^\top \mathbf{e}_i$ cover the item table (padding logit masked to $-\infty$), and the loss sums CE over all valid next-item targets in the packed history —

equivalent in target count to RecBole’s prefix-augmented CE [30], in one forward pass:

$$\mathcal{L}_{\text{SR}} = - \sum_u \sum_{k: i_{k+1} \neq 0} \log \frac{\exp(\ell_{u,k}(i_{k+1}))}{\sum_{j=1}^{|T|} \exp(\ell_{u,k}(j))} + \lambda_2 \|\Theta\|_2^2. \quad (6)$$

Any gap versus the upstream TiSASRec (BCE) baseline is therefore a Stage-1 training effect, not evidence for the language agents (Section V).

c) Role in the multi-agent pipeline.: $\pi_u^{(1)}$ is not a disposable candidate list for an unconstrained LLM. It is the numeric prior that the Guarded Reranking Agent may only softly blend with a listwise LLM proposal inside a top- K pool, under a learned trust weight α_u (Eqs. (17)–(18)). That evidence contract — not a new attention mechanism — is what distinguishes this agent from an off-the-shelf TiSASRec recommender.

C. Aspect-Sentiment Extraction Agent

This agent turns a review r into a set of triples $(a, \text{op}, \text{pol}, c)$, where a is an aspect span, op an opinion span, $\text{pol} \in \{-1, 0, 1\}$ the polarity, and $c \in [0, 1]$ a confidence. The *main* worker is a local LLM agent: it reads r in natural language and returns a candidate set $L(r)$. We choose an LLM for this role because free-text reviews often mix implicit aspects, hedges, and domain slang that a fixed supervised tagger may miss, and because the same local serving stack later drafts T_u and reranks the shortlist. At the same time, unconstrained generation can fail — the call may time out, return empty JSON, or invent spans that cannot be grounded in r . Leaving the pipeline with no triples in those cases would starve profiling and the manifesto of evidence. That is the justification for keeping a supervised **PyABSA** [31] ATEPC model (multilingual DeBERTa) as a narrow fallback, not as a co-equal proposer. PyABSA produces a classical set

$$C(r) = \{(a_j, \text{op}_j, \text{pol}_j, c_j)\}_{j=1}^{m(r)}, \quad (7)$$

with $m(r)$ the number of predictions. Let $\Phi_\tau(S) = \{(a, \text{op}, \text{pol}, c) \in S : c \geq \tau\}$ keep only triples whose confidence is at least τ . The selection rule makes the LLM primary and PyABSA secondary:

$$\text{ABSA}(r) = \Phi_{\tau_{\min}}(S(r)), \quad S(r) = \begin{cases} L(r) & \text{LLM usable,} \\ C(r) & \text{otherwise,} \end{cases} \quad (8)$$

with threshold $\tau_{\min} = 0.5$. “Usable” means a non-empty, well-formed extraction after the retry/repair budget in Section III-G; only then does the agent open $C(r)$. In the common case, therefore, the Aspect-Sentiment Extraction Agent is an LLM agent whose output is constrained by structure (typed triples) and by a confidence filter — not an unconstrained free-text summary, and not a PyABSA-first pipeline with LLM validation. Figure 3 shows the control flow and the PyABSA stack that supplies the fallback $C(r)$, adapted from Yang et al. [31]. This agent uses the smaller of our two self-hosted models (Section III-G). Table I summarizes the resulting corpus statistics.

TABLE I
CORPUS-LEVEL ABSA STATISTICS ON THE REVIEW-BEARING BENCHMARKS (LLM-PRIMARY EXTRACTION WITH PYABSA FALLBACK).

Category	Reviews	Triples	Triples/rev.	Pos. / Neu. / Neg. (%)
Beauty & P.C.	192,144	575,300	2.99	62.7 / 19.2 / 18.1
Sports & Out.	19,659	46,305	2.36	71.3 / 9.2 / 19.6
Toys & Games	105,491	333,608	3.16	74.6 / 7.7 / 17.6
Yelp (reviews)	465,422	415,145	0.89	77.0 / 13.6 / 9.4

D. Dynamic User-Profiling Agent

Running $\text{ABSA}(\cdot)$ over the user’s earlier reviews produces a stream of aspect mentions $\{(a_k, \text{pol}_k, t_k)\}$, each carrying an aspect, a polarity, and the timestamp of the review it came from. Those mentions usually come from the LLM extractor; what happens next is deliberately *not* left to another free-form LLM call. The profiling agent turns the stream into a time-decayed salience weight over aspects at any query time t by a closed-form rule. The unnormalized interest in aspect a is

$$I_u(a, t) = \sum_{t_k < t, a_k = a} |\text{pol}_k| e^{-\lambda(t-t_k)}, \quad (9)$$

where $\lambda > 0$ is a per-day decay rate and only mentions with $t_k < t$ are counted, so nothing from the future leaks into the query. The absolute polarity $|\text{pol}_k|$ measures how strongly the user cares about the aspect, not whether the mention was praise or complaint — a sharp complaint about “battery life” raises salience just as a strong compliment would. Normalizing over aspects mentioned before t gives

$$w_u(a, t) = \frac{I_u(a, t)}{\sum_{a'} I_u(a', t)}, \quad (10)$$

or the empty (all-zero) vector when the user has no prior aspect evidence. In the constrained-agent design, Eq. (10) is the algorithm that decides *which* aspects matter and *how much*; the LLM is not asked to invent those weights. Directional polarity remains available downstream through the signed values pol_k and through the review snippets read by the manifesto and rerank agents; a signed variant of (9) used in an earlier graph-based iteration is not part of the LLM-rerank path evaluated here.

E. Preference-Manifesto Agent

The manifesto agent compresses $w_u(\cdot, t_q)$ and recent evidence into one short sentence T_u . Here again the LLM does the language work — writing a readable preference statement — but under a hard content constraint: it may verbalize only the top k_{asp} aspects already ranked by Eq. (10), together with the user’s most recent non-empty reviews (or item metadata on ID-only tracks such as Yelp_AC; Section IV). The local 7B model is asked for a single sentence — for example, “the user currently prioritizes scent and long-wear . . .” — rather than to rediscover aspects from scratch. If that call fails, a deterministic template lists the same top aspects and recent items

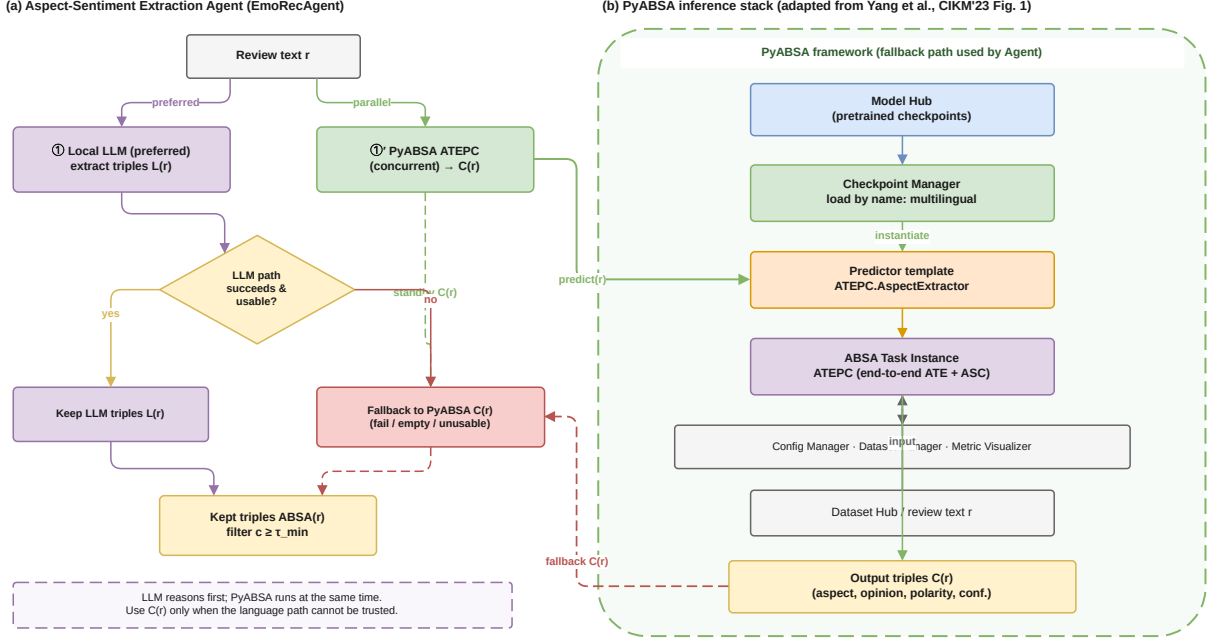


Fig. 3. Interaction between the Aspect-Sentiment Extraction Agent and PyABSA. (a) Agent control flow: the local LLM is the primary extractor and does the main work; PyABSA supplies $C(r)$ only when the LLM fails or yields an unusable extraction. (b) PyABSA inference stack used as that fallback (adapted from Yang et al. [31], Fig. 1): Checkpoint Manager loads the multilingual ATEPC predictor, which returns structured triples to the agent.

instead. Let T_u^{LLM} and T_u^{tp1} denote those two outputs. Then

$$T_u = \begin{cases} T_u^{\text{LLM}} & \text{LLM succeeds,} \\ T_u^{\text{tp1}} & \text{LLM fails, evidence exists,} \\ \emptyset & \text{no evidence,} \end{cases} \quad (11)$$

so whenever the user has qualifying review or metadata evidence, the Guarded Reranking Agent still receives a usable T_u . Results are cached per (u, t_q) so evaluation does not re-call the LLM for every candidate.

F. Guarded Reranking Agent

This agent is the synthesis step $\mathcal{G}$ in Eq. (2). The technical contribution is a *learned soft trust* weight $\alpha_u \in (0, 1)$ that blends the sequential prior $\pi_u^{(1)}$ with an LLM listwise proposal, rather than a hard accept/reject of the entire merge. Upstream steps only prepare a short evidence pack; the LLM proposes an order inside a bounded pool; g_ψ then decides *how much* to trust that proposal. Algorithm 1 summarizes the procedure.

a) *Evidence for the LLM (compressed).*: If $T_u = \emptyset$, we set $\hat{\pi}_u = \pi_u^{(1)}$ and stop. Otherwise a frozen sentence encoder [32] maps T_u to $\mathbf{t}_u$; an Alignment MLP f_ϕ projects it into the sequential space,

$$\mathbf{p}_u = f_\phi(\mathbf{t}_u) = \text{ELU}(W_\phi \mathbf{t}_u + \mathbf{b}_\phi), \quad (12)$$

trained with InfoNCE [33] against the next-item embedding (frozen item table $\mathbf{E}_I$, temperature τ , N_{neg} negatives):

$$\mathcal{L}_{\text{align}} = -\log \frac{\exp(\mathbf{p}_u^\top \mathbf{e}_{i+}/\tau)}{\exp(\mathbf{p}_u^\top \mathbf{e}_{i+}/\tau) + \sum_{i-} \exp(\mathbf{p}_u^\top \mathbf{e}_{i-}/\tau)}. \quad (13)$$

We form $\mathbf{x}_u = \omega \mathbf{s}_u + (1 - \omega) \mathbf{p}_u$ with backbone/alignment mix $\omega=0.7$ (and $\omega=1$ if alignment is unavailable), take the sequential pool $\mathcal{B}_u = \pi_u^{(1)}[K]$ ($K=100$), and lightly re-score candidates by

$$\text{score}(i) = \mathbf{e}_i^\top \mathbf{x}_u + \beta \sum_{i' \in \mathcal{A}_u^{\text{anc}}} \text{co}(i', i), \quad (14)$$

with a small co-occurrence boost $\beta=0.05$. A capped sub-pool of size $C=40$ is packed with T_u , scores, $\pi_u^{(1)}$ ranks, co-occurrence hints, and alignment confidence $c_u = \frac{1}{2}(1 + \cos(\mathbf{s}_u, \mathbf{p}_u))$. The local LLM returns a permutation π' of that sub-pool (numeric order on parse failure); merging with the untouched tail yields

$$\pi_u^{\text{m}} = \pi' \parallel (\pi_u^{(1)} \setminus \pi'). \quad (15)$$

b) *Soft trust blend (core).*: Hard accept/reject — keep all of π_u^{m} or none — wastes intermediate confidence and forces a brittle threshold. Fixed position-stability rules such as $(N, M) = (5, 10)$ likewise hard-code how much displacement is tolerable. We instead learn a continuous trust weight. Writing $\pi^{(1)} = \pi_u^{(1)}$ and $\pi^{\text{m}} = \pi_u^{\text{m}}$, define

$$\mathbf{z}_u = [c_u, \sigma(s_u(\pi^{(1)}(1)) - s_u(\pi^{(1)}(N_0))), \ell(T_u), \delta_u^{\text{max}}, \bar{\delta}_u, \rho_u]^\top, \quad (16)$$

Algorithm 1 Guarded Reranking Agent for user u at query time t_q

Require: $\pi_u^{(1)}$, s_u , T_u (may be empty), pool K , cap C , mix ω , head g_ψ

- 1: **if** T_u is empty **then**
- 2: **return** $\pi_u^{(1)}$
- 3: **end if**
- 4: $\mathbf{p}_u \leftarrow f_\phi(\text{SENTENCEENCODE}(T_u)); \mathbf{x}_u \leftarrow \omega \mathbf{s}_u + (1 - \omega) \mathbf{p}_u$
- 5: $\mathcal{B}_u \leftarrow \pi_u^{(1)}[K];$ score via Eq. (14); $\mathcal{B}_u^{\text{sub}} \leftarrow \text{top-}C$ by score
- 6: $\pi' \leftarrow \text{LOCALLLMRERANK}(\text{evidence pack}); \pi_u^{\text{m}} \leftarrow \pi' \parallel (\pi_u^{(1)} \setminus \pi')$
- 7: build $\mathbf{z}_u$; $\alpha_u \leftarrow g_\psi(\mathbf{z}_u)$ {Eq. (17)}
- 8:
- 9: **return** soft blend of π_u^{m} and $\pi_u^{(1)}$ via Eq. (18)

with $N_0=5$, $M_0=10$, normalized manifesto length $\ell(T_u)$, max/mean downward shifts $\delta_u^{\text{max}}, \bar{\delta}_u$ of the sequential top- N_0 under π^{m} , and ρ_u the fraction of those head items that remain in the merged top- M_0 . A logistic head maps features to trust,

$$\alpha_u = g_\psi(\mathbf{z}_u) = \sigma(\mathbf{w}_\psi^\top \mathbf{z}_u + b_\psi) \in (0, 1). \quad (17)$$

Inside the top- K pool we blend reciprocal-rank scores and re-sort; the tail beyond K is left unchanged:

$$\begin{aligned} \phi(\pi, i) &= 1/\text{rank}(i, \pi), \\ \tilde{s}_u(i) &= \alpha_u \phi(\pi^{\text{m}}, i) + (1 - \alpha_u) \phi(\pi^{(1)}, i) \quad (i \in \mathcal{B}_u), \\ \hat{\pi}_u &= \text{argsort}_{i \in \mathcal{B}_u}^\downarrow \tilde{s}_u(i) \parallel (\pi^{(1)}[K:]). \end{aligned} \quad (18)$$

Thus $\alpha_u \rightarrow 1$ recovers the LLM merge, $\alpha_u \rightarrow 0$ recovers the sequential prior, and hard accept/reject is the degenerate special case $\hat{\pi}_u = \pi^{\text{m}}$ iff $\alpha_u \geq \theta$ else $\pi^{(1)}$ (used only as an ablation baseline in Section V-E). Because only the top- K head is touched, HR@ k and Recall@ k for every $k \geq K$ are unchanged by the blend.

c) Training g_ψ . We train g_ψ offline on validation queries without updating the Sequential Ranking Agent, f_ϕ , or the LLM. Labels mark whether the pure LLM merge improves the held-out next item,

$$y_u = \mathbb{I}[\text{rank}(i_u^*, \pi_u^{\text{m}}) < \text{rank}(i_u^*, \pi_u^{(1)})], \quad (19)$$

and the head minimizes binary cross-entropy

$$\mathcal{L}_{\text{gate}} = - \sum_u (y_u \log \alpha_u + (1 - y_u) \log(1 - \alpha_u)). \quad (20)$$

At test time g_ψ is frozen and Eq. (18) uses the continuous α_u (no decision threshold). An optional ceiling $\delta_u^{\text{max}} \leq \Delta_{\text{max}}$ (default 20) may zero α_u for pathological permutations; it is a safety net, not the primary rule.

Figure 4 sketches the flow: compute evidence $\rightarrow$ LLM proposes a shortlist order $\rightarrow$ soft-trust blend with $\alpha_u = g_\psi(\mathbf{z}_u)$.

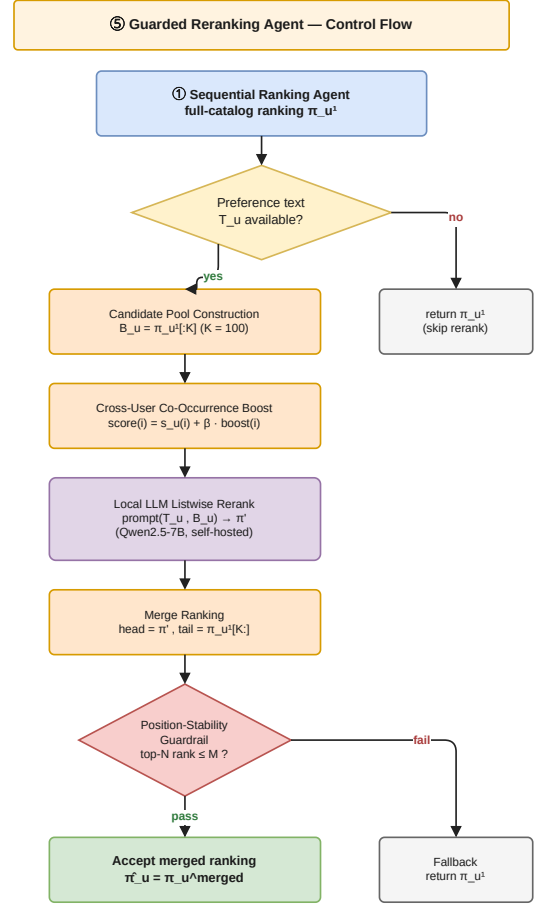


Fig. 4. Guarded Reranking Agent (5): aligned T_u and pool scores are packed for the local LLM; the LLM proposes a listwise order; a learned soft trust $\alpha_u = g_\psi(\mathbf{z}_u)$ blends that proposal with $\pi_u^{(1)}$ (Eq. (18)).

G. Self-Hosted Local LLM Serving Layer

All LLM calls — primary ABSA extraction, manifesto drafting, and listwise rerank — go through an OpenAI-compatible endpoint running Hugging Face Text Generation Inference (TGI) inside the operator’s own infrastructure. No request leaves that boundary. We split the work across two open-weight sizes: **Qwen2.5-3B-Instruct** for the Aspect-Sentiment Extraction Agent, where the job is a short, mostly extractive judgment; and **Qwen2.5-7B-Instruct** for the manifesto and listwise rerank, which need a longer context and a coherent sentence or permutation. Both run at temperature 0 with three retries and a structured-output repair layer for truncated JSON. These retries are part of keeping the LLM on the main path: only after they are exhausted does ABSA open the PyABSA fallback (Section III-C). Likewise, if rerank parsing fails, the agent sets $\alpha_u=0$ and keeps the numeric sequential order (Section III-F). Because Eqs. (17)–(18) depend on merge features and g_ψ rather than on which model proposed π' , a different self-hosted model can be swapped in without changing the soft-trust rule. The Alignment MLP, sentence encoder, and trust head all run locally, so T_u never leaves the operator boundary on its

TABLE II
DATASET STATISTICS AFTER FILTERING AND SPLITTING. “PVT” IS THE FRACTION OF TEST-SPLIT INTERACTIONS CARRYING A VERIFIED-PURCHASE REVIEW (AMAZON ONLY; YELP HAS NO EQUIVALENT FIELD, HENCE 0).

Benchmark	Users	Items	Inter.	Train	Valid	Test	Split / PVT
Beauty & P.C.	4,976	7,985	183,326	144,653	16,058	22,615	chrono. / 0.422
Sports & Out.	1,403	1,017	19,489	15,030	1,565	2,894	chrono. / 0.955
Toys & Games	4,523	6,712	103,996	81,419	8,497	14,080	chrono. / 0.413
Yelp (reviews)	4,904	9,475	432,138	343,776	41,034	47,328	chrono. / 0.000
Yelp_AC	30,449	20,038	316,541	255,643	30,449	30,449	LOO / —

way into $\mathbf{p}_u$ or $\mathbf{z}_u$.

On the critical path, the Sequential Ranking Agent needs only one embedding lookup and one dot product per item (Eq. (5)). The Guarded Reranking Agent calls the LLM at most once per query (at most $C = 40$ candidates), and ABSA at most twice per historical review. Both language results are cached per review or per (u, t_q) , so full-catalog evaluation does not re-query the LLM for every candidate.

IV. EXPERIMENTAL SETUP

A. Datasets

We evaluate on five benchmarks from two data families, summarized in Table II. Three are Amazon Reviews 2023 categories (*Beauty & Personal Care*, *Sports & Outdoors*, *Toys & Games*), self-curated with a 10-core user/item filter, a chronological 80/10/10 train/valid/test split, and a minimum history length of 5. Two are Yelp tracks built to answer different questions: *Yelp* keeps the native review text and a 20-core filter with the same chronological split, so that its aspect-sentiment and cross-user-boost machinery can be exercised exactly as on the Amazon categories; *Yelp_AC* instead reproduces the AC-TSR benchmark’s own preprocessing recipe [9] — 5-core filtering, transactions restricted to the window after 1 January 2019, and a leave-one-out (last-item test, second-to-last-item validation) split — specifically so that its sequential-ranking numbers can be checked against an externally published TiSASRec result on the same data recipe (Section V). *Yelp_AC* carries no review text into the Guarded Reranking Agent; its preference manifesto T_u is instead generated from item metadata (name, category) rather than from ABSA triples, and its cross-user lookup uses co-visitation rather than a review-gated anchor set.

B. Baselines

We compare against classical collaborative-filtering, sequential, and LLM-agent baselines, all trained and evaluated on the same processed splits under the protocol below. **LightGCN** [13], **XSimGCL** [15], and **CORE** [16] represent, respectively, plain graph convolution, contrastive graph convolution, and consistent-representation session encoding. **TiSASRec** [12] (trained with the original BCE objective, upstream PyTorch implementation) is the closest published time-interval self-attention baseline to our Sequential Ranking Agent and the strongest purely

neural external baseline; our agent shares that design lineage but is trained separately with dense full-catalog CE (Section III-B). **Agent4Rec** [23] is included as an LLM-agent baseline, repurposed from a user simulator into a direct ranker over the same candidate space. On the *Yelp_AC* track, we additionally reproduce **TiSASRec trained with a cross-entropy objective via RecBole** [30], following the AC-TSR fork’s own training recipe, so that our reproduction can be checked against the number the AC-TSR paper itself reports for TiSASRec on the same benchmark (Section V).

We additionally report two internal, non-LLM ablation points where available: **SR-only** is our Sequential Ranking Agent alone (the deep time-interval encoder of Section III-B) with the Guarded Reranking Agent disabled, isolating exactly what the guarded LLM rerank adds; and, on Beauty, an earlier graph-based variant (**HGT, no agent**) that replaces the sequential backbone with a heterogeneous graph transformer over an aspect co-occurrence graph but without any LLM stage, kept here as a non-LLM point of reference from an earlier iteration of this project. Two vendored baselines, **AgentCF** [24] and **SimGCL** [14], are implemented in our repository but have not yet completed a full run at submission time; we report this explicitly as an open item in Section VI rather than omit it silently.

C. Evaluation Protocol and Metrics

All Amazon-category and Yelp (reviews) results use *Protocol B*: full-catalog ranking (every item in $\mathcal{I}$ is scored for every test interaction, not a sampled subset), aggregated with the **user_batch** evaluation procedure and **user_mean** aggregation across users. We additionally report a sampled-negative variant on Beauty (100 sampled negatives per positive) for direct comparability with baselines that are commonly evaluated that way in the literature. *Yelp_AC* uses the leave-one-out full-ranking protocol native to RecBole, matching AC-TSR’s own evaluation setup exactly. For a ranked list π and a ground-truth relevant set $\mathcal{R}$ (a single held-out item, for all five benchmarks), we report, at cutoff k ,

$$\text{HR}@k = \mathbb{1}[\exists i \leq k : \pi_i \in \mathcal{R}], \quad (21)$$

$$\text{Recall}@k = \frac{|\mathcal{R} \cap \pi_{:k}|}{|\mathcal{R}|}, \quad (22)$$

$$\text{NDCG}@k = \frac{1}{\text{IDCG}@k} \sum_{p=1}^k \frac{2^{\mathbb{1}[\pi_p \in \mathcal{R}]} - 1}{\log_2(p+1)}, \quad (23)$$

$$\text{MRR}@k = \frac{1}{\text{rank}_1(\pi, \mathcal{R})} \text{ if } \text{rank}_1(\pi, \mathcal{R}) \leq k \text{ else } 0, \quad (24)$$

where $\text{rank}_1(\pi, \mathcal{R})$ is the position of the first relevant item in π . With a single relevant item per query, $\text{HR}@k$ and $\text{Recall}@k$ coincide; we report both because different baselines in the literature conventionally use one or the other. We additionally report $\text{AvgHR}@\{1, 3, 5\} =$

$\frac{1}{3} \sum_{k \in \{1,3,5\}} \text{HR}@k$ as a single head-of-list summary statistic, following the convention used elsewhere in the agentic-recommendation literature [25].

D. Implementation Details

The Sequential Ranking Agent uses hidden size 64, 2 self-attention blocks, 1 attention head, dropout 0.2, ℓ_2 embedding penalty 10^{-4} , a maximum sequence length of 50, and a time-span of 256 discretized buckets; it is trained with dense full-catalog CE (Eq. (6)) under Adam ($\text{lr} = 10^{-3}$, batch size 2048) for up to 1000 epochs with early stopping (patience 10) on validation link $\text{HR}@10$. The Guarded Reranking Agent uses pool size $K=100$, sub-pool cap $C=40$, cross-user boost $\beta=0.05$, backbone/alignment mix $\omega=0.7$, Alignment MLP trained with InfoNCE (τ grid $\{0.05, 0.07, 0.1\}$, $N_{\text{neg}}=32$), and a soft-trust head g_ψ (logistic α_u on Eq. (16), trained with Eq. (20); continuous blend via Eq. (18)), with time-decay rate $\lambda=0.01$ per day for the profiling agent. All local LLM inference uses Qwen2.5-3B-Instruct (ABSA) and Qwen2.5-7B-Instruct (listwise rerank) served through TGI 3.2.1 at temperature 0. Guarded-rerank evaluation dispatches up to 8 concurrent users against the rerank endpoint; ABSA extraction runs with 32 concurrent workers. All hyperparameters above are configuration defaults, not per-benchmark-tuned values, and are identical across the three Amazon categories and Yelp (reviews); Yelp_AC additionally trains a RecBole cross-entropy sequential-ranking following the AC-TSR fork’s own hyperparameter search space for a fair, literature-matched comparison.

V. RESULTS AND DISCUSSION

We report results in six parts. We begin with the Beauty & Personal Care benchmark, where the full baseline suite is available, and then examine how the guarded-rerank stage behaves on the other review-bearing domains. We next verify our Sequential Ranking Agent against the published Yelp_AC numbers of AC-TSR [9], ablate the language reranker versus any guardrail, compare the soft-trust blend against hard accept/reject and a fixed (N, M) rule, and finally discuss what the current evidence does and does not support for local LLM deployment.

A. Overall Performance on Beauty & Personal Care

Table III summarizes the full-catalog Protocol B comparison on Beauty & Personal Care. We make the following observations.

Among the external baselines, TiSASRec (BCE) is the strongest sequential model, while LightGCN, XSimGCL, and CORE form a lower but competitive graph/session group. Agent4Rec, which we use as an LLM-agent reference, is close to this group and does not surpass TiSASRec under full-catalog ranking. This is consistent with our design premise: a general-purpose agent is not by itself a substitute for a model trained to score the entire catalog.

The Sequential Ranking stage already improves over all external baselines reported here. Relative to the upstream

TABLE III
BEAUTY & PERSONAL CARE, FULL-CATALOG PROTOCOL B
(USER_BATCH/USER_MEAN). BEST IN BOLD. DASHES (—) MARK RUNS NOT YET COMPLETED.

Method	AvgHR _{1,3,5}	HR@1	HR@10	HR@20	NDCG@10	NDCG@20
LightGCN [13]	0.0234	0.0100	0.0595	0.0945	0.0127	0.0165
SimGCL [14]	—	—	—	—	—	—
XSimGCL [15]	0.0259	0.0102	0.0723	0.1115	0.0150	0.0195
CORE [16]	0.0251	0.0100	0.0711	0.1276	0.0139	0.0203
GRU4Rec [34]	—	—	—	—	—	—
SASRec [10]	—	—	—	—	—	—
BERT4Rec [11]	—	—	—	—	—	—
SSE-PT [35]	—	—	—	—	—	—
TiSASRec (BCE) [12]	0.0281	0.0098	0.0762	0.1324	0.0143	0.0208
Agent4Rec [23]	0.0272	0.0123	0.0683	0.1168	0.0149	0.0202
AgentCF [24]	—	—	—	—	—	—
EmoRecAgent SR-only	0.0375	0.0145	0.1047	0.1648	0.0202	0.0269
EmoRecAgent (full)	0.0503	0.0297	0.1162	0.1714	0.0262	0.0321

TABLE IV
BEAUTY & PERSONAL CARE, 100-SAMPLED-NEGATIVE PROTOCOL.
“THR.” = USERS/S.

Method	HR@10	HR@20	AvgHR _{1,3,5}	thr.
LightGCN [13]	0.3167	0.4918	0.1370	311.0
XSimGCL [15]	0.3331	0.4808	0.1511	324.9
CORE [16]	0.4255	0.6117	0.1881	212.2
TiSASRec (BCE) [12]	0.4394	0.6148	0.1963	211.8
Agent4Rec [23]	0.3349	0.5100	0.1531	—
HGT, no agent (ours)	0.4105	0.5952	0.1764	100.0
EmoRecAgent (full)	—	—	—	—

TiSASRec (BCE) baseline, $\text{HR}@10$ rises from 0.0762 to 0.1047, and $\text{NDCG}@10$ from 0.0143 to 0.0202; we treat this gap as largely a backbone-training / implementation difference (dense full-catalog CE and our training recipe versus the BCE baseline), not as evidence for the language stages. The contribution we emphasize is the further lift from review-conditioned Guarded Reranking on top of the *same* backbone: $\text{HR}@10$ increases to 0.1162 (+11.0% over sequential ranking), $\text{AvgHR}@\{1, 3, 5\}$ to 0.0503 (+34.1%), and $\text{HR}@1$ nearly doubles from 0.0145 to 0.0297. In short, the sequential backbone carries most of the ranking quality on Beauty, while the local language stage provides an additional, non-trivial head-of-list lift.

Table IV reports the complementary protocol with 100 sampled negatives, which remains common in sequential-recommendation studies. Here TiSASRec attains the best accuracy ($\text{AvgHR}@\{1, 3, 5\}=0.1963$) at 211.8 users/s. CORE is close in accuracy; LightGCN and XSimGCL are faster (>300 users/s) but less accurate. Our earlier HGT variant, without the LLM stage, reaches 0.1764 at 100 users/s. Because sampling makes the ranking task much easier, these numbers are not directly comparable with Table III. We therefore evaluate the complete EmoRecAgent pipeline only under full-catalog ranking and omit it from Table IV.

B. Performance across Review-Bearing Domains

We next ask whether the Beauty result transfers to the other review-bearing benchmarks. Tables V–VII mirror the Beauty layout for Sports & Outdoors, Toys & Games, and Yelp; cells still missing a completed run are left as dashes. Table VIII compresses the sequential-ranking

TABLE V

SPORTS & OUTDOORS, FULL-CATALOG PROTOCOL B. DASHES MARK RUNS NOT YET COMPLETED.

Method	AvgHR _{1,3,5}	HR@1	HR@10	HR@20	NDCG@10	NDCG@20
LightGCN [13]	—	—	—	—	—	—
SimGCL [14]	—	—	—	—	—	—
XSimGCL [15]	—	—	—	—	—	—
CORE [16]	—	—	—	—	—	—
GRU4Rec [34]	—	—	—	—	—	—
SASRec [10]	—	—	—	—	—	—
BERT4Rec [11]	—	—	—	—	—	—
SSE-PT [35]	—	—	—	—	—	—
TiSASRec (BCE) [12]	—	—	—	—	—	—
Agent4Rec [23]	—	—	—	—	—	—
AgentCF [24]	—	—	—	—	—	—
EmoRecAgent SR-only	0.0539	0.0221	0.1226	0.1839	0.0415	—
EmoRecAgent (full)	0.0836	0.0527	0.1475	0.1996	0.0592	—

TABLE VI

TOYS & GAMES, FULL-CATALOG PROTOCOL B. DASHES MARK RUNS NOT YET COMPLETED.

Method	AvgHR _{1,3,5}	HR@1	HR@10	HR@20	NDCG@10	NDCG@20
LightGCN [13]	—	—	—	—	—	—
SimGCL [14]	—	—	—	—	—	—
XSimGCL [15]	—	—	—	—	—	—
CORE [16]	—	—	—	—	—	—
GRU4Rec [34]	—	—	—	—	—	—
SASRec [10]	—	—	—	—	—	—
BERT4Rec [11]	—	—	—	—	—	—
SSE-PT [35]	—	—	—	—	—	—
TiSASRec (BCE) [12]	—	—	—	—	—	—
Agent4Rec [23]	—	—	—	—	—	—
AgentCF [24]	—	—	—	—	—	—
EmoRecAgent SR-only	—	0.0102	0.0608	0.0988	0.0157	—
EmoRecAgent (full)	0.0237	0.0093	0.0612	0.0988	0.0155	—

versus full-pipeline comparison, and Figure 5 visualizes the corresponding relative HR@10 lift.

The largest improvement appears on Sports. AvgHR@{1, 3, 5} rises from 0.0539 to 0.0836 (+55.0%), and HR@1 from 0.0221 to 0.0527 (+138.5%). Sports also has the smallest catalog (1,017 items) and the densest verified-review coverage (95.5%) among our Amazon categories. A plausible reading is that abundant review evidence is more useful when the candidate pool is already small and relatively coherent.

Toys tells a different story. At HR@10 and HR@20 the pipeline is essentially unchanged (0.0608→0.0612; 0.0988 unchanged), while HR@1 and NDCG@10 decline slightly. The language stage therefore contributes little, and occasionally hurts the very top of the list. On Yelp the picture is likewise mixed: HR@3 improves slightly (0.0886→0.0893), but HR@10 decreases from 0.2392 to 0.2337. Yelp also yields the strongest Sequential Ranking Agent among the four review-bearing domains, so a second-stage reorder has less room to help.

Overall, review-conditioned reranking is helpful on Beauty and Sports, nearly neutral on Toys, and mildly mixed on Yelp. The benefit is therefore domain dependent rather than automatic. Until the remaining baseline rows in Tables V–VII are filled, these tables should be read as evidence of this variation, not as a saturated cross-domain leaderboard.

C. Reproducibility against the Yelp_AC Benchmark

To check that our Sequential Ranking Agent is not benefiting from an easier data recipe, we reproduce the Yelp_AC setup of AC-TSR [9]: 5-core filtering, interactions after January 2019, and leave-one-out evaluation.

TABLE VII

YELP (REVIEWS), FULL-CATALOG PROTOCOL B. DASHES MARK RUNS NOT YET COMPLETED.

Method	AvgHR _{1,3,5}	HR@1	HR@10	HR@20	NDCG@10	NDCG@20
LightGCN [13]	—	—	—	—	—	—
SimGCL [14]	—	—	—	—	—	—
XSimGCL [15]	—	—	—	—	—	—
CORE [16]	—	—	—	—	—	—
GRU4Rec [34]	—	—	—	—	—	—
SASRec [10]	—	—	—	—	—	—
BERT4Rec [11]	—	—	—	—	—	—
SSE-PT [35]	—	—	—	—	—	—
TiSASRec (BCE) [12]	—	—	—	—	—	—
Agent4Rec [23]	—	—	—	—	—	—
AgentCF [24]	—	—	—	—	—	—
EmoRecAgent SR-only	—	0.0342	0.2392	0.3747	0.0331	—
EmoRecAgent (full)	0.0858	0.0338	0.2337	0.3697	0.0330	—

TABLE VIII

COMPACT CROSS-DOMAIN SUMMARY: SEQUENTIAL RANKING VS. FULL EMOREAGENT (FULL-CATALOG PROTOCOL B).

Benchmark	Method	AvgHR _{1,3,5}	HR@1	HR@10	HR@20	NDCG@10
Sports & Out.	SR-only	0.0539	0.0221	0.1226	0.1839	0.0415
	EmoRecAgent	0.0836	0.0527	0.1475	0.1996	0.0592
Toys & Games	SR-only	—	0.0102	0.0608	0.0988	0.0157
	EmoRecAgent	0.0237	0.0093	0.0612	0.0988	0.0155
Yelp (reviews)	SR-only	—	0.0342	0.2392	0.3747	0.0331
	EmoRecAgent	0.0858	0.0338	0.2337	0.3697	0.0330

Table IX shows that our RecBole TiSASRec run reaches Recall@10=0.0614 and NDCG@10=0.0384, within 0.6% and 0.8% of the TiSASRec numbers reported by AC-TSR (0.0618 and 0.0387). The match is close enough that we regard the reproduction as faithful.

AC-TSR’s attention-calibrated variant further improves these scores to 0.0664 / 0.0407 by adjusting the sequential encoder itself. EmoRecAgent instead leaves the encoder intact and applies a review-conditioned reorder on top of it. The two ideas are complementary rather than competing, and combining them is a natural direction for future work. Table IX reserves rows for the remaining baselines and for the full EmoRecAgent pipeline; unfinished cells are left as dashes rather than estimated from other domains.

D. Effect of the Language Reranker and Guardrail

The previous tables compare sequential ranking with the complete system, but do not isolate the language call from the guardrail. To separate the two, we evaluate three variants on a fixed Yelp subset: *No-LLM*, which keeps the cross-user boost and falls back to the numeric order; *LLM*, *no guardrail*, which accepts the language model’s merged list as-is; and *LLM + soft trust*, our default configuration, which restricts the permutation to a bounded pool and blends it with $\pi_u^{(1)}$ under Eqs. (17)–(18).

Table X shows that the default configuration is best on HR@10, NDCG@10, and AvgHR@{1, 3, 5}. Relative to No-LLM, it raises AvgHR@{1, 3, 5} from 0.0918 to 0.1011 (+10.1%), indicating that the language call itself contributes beyond the co-occurrence boost. Relative to the unguarded language reranker, it improves the same metric from 0.0978 to 0.1011 and recovers most of the drop in HR@20 (0.3912→0.4072). No-LLM remains slightly higher at HR@20 (0.4092), so the guardrail is not uniformly superior at every cutoff. The practical message

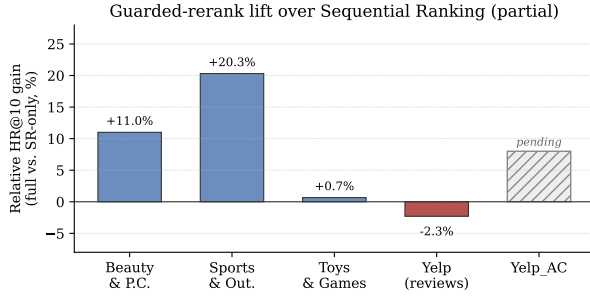


Fig. 5. Relative HR@10 gain of the full pipeline over sequential ranking. Measured domains are filled; Yelp_AC is marked pending until the full-pipeline run completes.

TABLE IX

YELP_AC (LEAVE-ONE-OUT, FULL RANKING). “AC-TSR (PAPER)” ROWS ARE NUMBERS REPORTED IN [9] ON THE SAME PREPROCESSING RECIPE, INCLUDED AS A LITERATURE-ANCHORED REFERENCE POINT, NOT RE-RUN BY US. DASHES MARK RUNS NOT YET COMPLETED.

Method	Recall@10	Recall@20	NDCG@10	NDCG@20
LightGCN [13]	—	—	—	—
SimGCL [14]	—	—	—	—
XSimGCL [15]	—	—	—	—
CORE [16]	—	—	—	—
GRU4Rec [34]	—	—	—	—
SASRec [10]	—	—	—	—
BERT4Rec [11]	—	—	—	—
SSE-PT [35]	—	—	—	—
TiSASRec, AC-TSR (paper) [9]	0.0618	0.0909	0.0387	0.0460
TiSASRec+AC, AC-TSR (paper) [9]	0.0664	0.0955	0.0407	0.0480
Agent4Rec [23]	—	—	—	—
AgentCF [24]	—	—	—	—
EmoRecAgent SR (RecBole CE, ours)	0.0614	0.0885	0.0384	0.0452
EmoRecAgent (full pipeline)	—	—	—	—

is nonetheless clear: unconstrained LLM reordering can damage the list, and bounding that reorder is preferable to accepting it wholesale. Absolute scores in Table X should not be compared with the full Yelp numbers in Table VIII, which use a different evaluation pass.

E. Soft Trust vs. Hard Accept/Reject and Fixed (N, M)

Table X shows that *some* constraint helps relative to an ungated LLM, but it does not isolate the soft-trust blend of Eqs. (17)–(18) from hard accept/reject or from a classical fixed position-stability rule. We therefore reserve a controlled comparison that holds the Sequential Ranking Agent, Alignment MLP, evidence pack, LLM backend, pool size K , and sub-pool cap C fixed, and varies only the post-LLM policy:

- *LLM, no guardrail*: always keep π_u^m ($\alpha_u \equiv 1$).
- *Fixed (N, M)*: hard accept iff every item in $\pi_u^{(1)}[: N]$ remains inside the merged top- M , with $(N, M) = (5, 10)$.
- *Hard learned gate*: keep π_u^m iff $\alpha_u = g_\psi(\mathbf{z}_u) \geq \theta$ (degenerate case of Eq. (17)).
- *Soft trust blend* (default): continuous blend via Eq. (18).

Table XI will report full-catalog Protocol B metrics on Beauty (primary) and, where available, the same Yelp subset as Table X. Dashes mark runs not yet completed; we do not interpolate missing cells. Once filled, this table is

TABLE X
GUARDED-RERANK ABLATION ON A FIXED YELP EVALUATION SUBSET (INTERNALLY COMPARABLE; NOT DIRECTLY COMPARABLE TO TABLE VIII).

Variant	HR@10	HR@20	NDCG@10	AvgHR@{1,3,5}
No-LLM (cross-user boost only)	0.2695	0.4092	0.0366	0.0918
LLM rerank, no guardrail	0.2675	0.3912	0.0366	0.0978
LLM + soft trust (deployed)	0.2715	0.4072	0.0377	0.1011

Local vs. API listwise rerank (same prompt / pool / guardrail) — pending

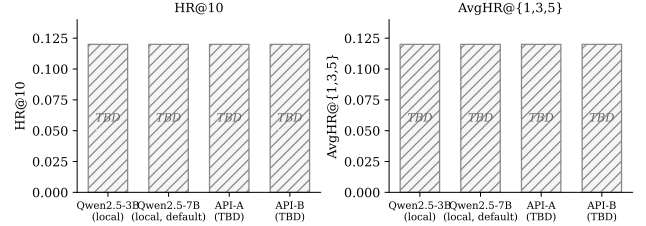


Fig. 6. Placeholder visualization for the local-vs-API comparison in Table XII. Bars are left empty until the controlled runs finish.

the direct empirical test of whether continuous soft trust improves over hard accept/reject and over the hand-set displacement bound.

F. Discussion of Local Deployment

A recurring deployment question is whether a small, locally hosted model is usable for the language stage under an operator-boundary constraint. Our evidence on this point is encouraging but incomplete.

On Beauty and Sports, the local reranker improves over the sequential backbone by a clear margin. The Yelp ablation likewise shows that removing the language call lowers top-rank accuracy. At the same time, Toys is nearly unchanged, and the full Yelp evaluation declines at several cutoffs. The language stage is therefore useful when the domain and review evidence support it, not merely because an LLM is present.

We do not claim parity with commercial API models. Table XII and Figure 6 reserve the controlled comparison still needed: the same prompt, candidate pool, alignment path, and soft-trust blend, with only the listwise-rerank backend changing. Until those cells are filled, the present experiments support a narrower claim — that a 3B/7B open-weight model served locally at temperature 0 can produce usable listwise output and, under the guarded interface, improve ranking on some domains without sending customer data outside the operator’s infrastructure.

VI. CONCLUSION

We presented EmoRecAgent, a multi-agent recommendation framework built around two requirements that remain under-addressed together in the LLM-recommendation literature: personalization that reads a customer’s own words to infer shifting preferences, and an operator-controlled inference boundary that never sends those words to a third party. The central design idea is

TABLE XI

POST-LLM POLICY ABLATION: SOFT TRUST BLEND VS. HARD LEARNED GATE VS. FIXED $(N, M) = (5, 10)$. PROMPT, POOL (K, C) , ALIGNMENT MLP, AND LLM BACKEND ARE HELD FIXED. DASHES MARK RUNS NOT YET COMPLETED. MEAN α_u IS REPORTED FOR SOFT TRUST; ACCEPT RATE FOR HARD POLICIES.

Domain	Post-LLM policy	AvgHR _{1,3,5}	HR@1	HR@10	HR@20	NDCG@10	Mean α_u / accept	Notes
Beauty	LLM, no guardrail	—	—	—	—	—	—	$\alpha_u \equiv 1$
	Fixed $(N, M) = (5, 10)$	—	—	—	—	—	—	—
	Hard learned gate	—	—	—	—	—	—	—
	Soft trust blend (default)	—	—	—	—	—	—	Eq. (18)
Yelp subset	LLM, no guardrail	—	—	—	—	—	—	cf. Table X
	Fixed $(N, M) = (5, 10)$	—	—	—	—	—	—	—
	Hard learned gate	—	—	—	—	—	—	—
	Soft trust blend (default)	—	—	—	—	—	—	Eq. (18)

TABLE XII

CONTROLLED LOCAL-VS-API LISTWISE RERANK ON BEAUTY (FULL-CATALOG PROTOCOL B). PROMPT, TOP- K POOL, ALIGNMENT MLP, AND SOFT-TRUST HEAD ARE HELD FIXED; ONLY THE MODEL BEHIND THE RERANK CALL CHANGES. DASHES MARK RUNS NOT YET COMPLETED.

Rerank backend	Serving	AvgHR _{1,3,5}	HR@1	HR@10	NDCG@10	Latency (ms)
None (SR-only)	—	0.0375	0.0145	0.1047	0.0202	—
Qwen2.5-3B-Instruct	local / TGI	—	—	—	—	—
Qwen2.5-7B-Instruct (default)	local / TGI	0.0503	0.0297	0.1162	0.0262	—
API model A (TBD)	remote API	—	—	—	—	—
API model B (TBD)	remote API	—	—	—	—	—

that LLM agents should do that language work, but not in isolation — each generative decision is bound to an algorithm or hard constraint. Five cooperating agents share the pipeline. The Aspect-Sentiment Extraction Agent uses a local LLM as the primary ABSA reasoner and keeps PyABSA only as a justified fallback when generation fails; profiling fixes w_u by a closed-form time-decay rule; the manifesto verbalizes those weights into T_u ; and the Guarded Reranking Agent aligns T_u with an InfoNCE-trained MLP, feeds the computed evidence pack to a local LLM for the shortlist decision, and blends that decision with $\pi_u^{(1)}$ under a learned soft trust $\alpha_u = g_\psi(\mathbf{z}_u)$. Across five benchmarks spanning Amazon Reviews 2023 and Yelp, the full system improved ranking quality over the *same* sequential backbone on Beauty & Personal Care (+11.0% HR@10) and Sports & Outdoors (+20.3% HR@10), changed little on Toys & Games, and produced mixed results on Yelp. The ablation further showed that the language reranker improves top-ranked accuracy on the evaluated subset and that soft-trust blending is preferable to accepting the LLM list unconditionally.

A. Limitations

We are explicit about five limitations. First, three of our five benchmarks (Sports, Toys, Yelp) and one entirely (Yelp_AC) currently report sequential-ranking-vs-full-pipeline comparisons without the full baseline suite used on Beauty; we report exactly the cells we have measured and leave the rest blank rather than interpolated, but the comparison table is not yet saturated. Second, we have not yet run a controlled local-vs-API-LLM ablation on the same prompts and benchmark (Section V-F); until that table is filled, we claim only that a self-hosted

3B/7B stack is *usable* under the guarded interface, not that it matches commercial API quality. Third, because the Aspect-Sentiment Extraction Agent puts the LLM on the main path, extraction quality still depends on generative judgment whenever that path succeeds; PyABSA covers only failure cases, and we have not yet run a dedicated human-annotated gold-label evaluation across all four review-bearing benchmarks (only Beauty currently has one). Fourth, the soft-trust head g_ψ is still a shallow logistic map trained with the proxy label in Eq. (19); Table X shows that constraining the LLM matters, but the direct comparison of soft blend vs. hard learned gate vs. fixed (N, M) vs. ungated LLM in Table XI is not yet filled, nor have we explored richer trust models or listwise-aware supervision of α_u . Fifth, the neutral-to-mixed result on Toys and Yelp (Table VIII) is, at present, observed but not fully explained; our working hypothesis — that the Guarded Reranking Agent’s benefit tracks review density and sequential-ranking head-room — is consistent with the data we have but would need a controlled variation of review coverage to establish causally.

B. Future Work

We plan to (i) complete the baseline suite (SAS-Rec, BERT4Rec, GRU4Rec, SSE-PT, and the already-vendored AgentCF and SimGCL) across all five benchmarks, so that every dashed cell in Tables III–VII and IX is replaced by a measured value; (ii) fill Table XII / Figure 6 with a controlled local-vs-API comparison that holds the prompt, pool, alignment path, and soft-trust head fixed and varies only the model behind the listwise rerank call; (iii) attach an attention-calibration module in the spirit of AC-TSR [9] to the Sequential Ranking Agent itself, which Table IX suggests is a complementary, largely orthogonal source of improvement to the listwise rerank; (iv) deepen soft trust (deeper head, listwise-aware targets for α_u , and joint training with f_ϕ); and (v) extend evaluation beyond the static, full-catalog Protocol B used throughout this paper to the interactive, tool-using evaluation setting exemplified by AgentRecBench [26], which would let us measure EmoRecAgent under evolving-interest and cold-start conditions that a single held-out next item cannot capture.

REFERENCES

- [1] Z. Zhao, W. Fan, J. Li, Y. Liu, X. Mei, Y. Wang, Z. Wen, F. Wang, X. Zhao, J. Tang, and Q. Li, "Recommender systems in the era of large language models (LLMs)," *IEEE Transactions on Knowledge and Data Engineering*, 2024.
- [2] S. Luo, B. He, H. Zhao, Y. Huang, A. Zhou, Z. Li, Y. Xiao, M. Zhang, and C. Wang, "Privacy in LLM-based recommendation: Recent advances and future directions," *arXiv preprint arXiv:2406.01363*, 2024.
- [3] J. He, Y. Gu, M.-C. Chen, and K. Chen, "Membership inference attacks on LLM-based recommender systems," *arXiv preprint arXiv:2508.18665*, 2025.
- [4] T. Khezresmaeilzadeh, J. Zhang, D. Andreadis, and K. Psounis, "Preserving privacy and utility in LLM-based product recommendations," *arXiv preprint arXiv:2505.00951*, 2025.
- [5] X. Huang, J. Lian, Y. Lei, J. Yao, D. Lian, and X. Xie, "Recommender AI agent: Integrating large language models for interactive recommendations," *ACM Transactions on Information Systems*, 2025, arXiv:2308.16505.
- [6] Y. Wang, Z. Jiang, Z. Chen, F. Yang, Y. Zhou, E. Cho, X. Fan, Y. Lu, X. Huang, and Y. Yang, "RecMind: Large language model powered agent for recommendation," in *Findings of the Association for Computational Linguistics: NAACL*, 2024, arXiv:2308.14296.
- [7] J. Gao, B. Chen, X. Zhao, W. Liu, X. Li, Y. Wang, Z. Wang, W. Li, H. Guo, and R. Tang, "LLM4Rerank: LLM-based auto-reranking framework for recommendations," in *Proceedings of the ACM Web Conference (WWW)*, 2025.
- [8] Z. Wang, Y. Yu, W. Zheng, W. Ma, and M. Zhang, "MACRec: A multi-agent collaboration framework for recommendation," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (Demo Track)*, 2024, arXiv:2402.15235.
- [9] P. Zhou, Q. Ye, Y. Xie, J. Gao, S. Wang, J. B. Kim, C. You, and S. Kim, "Attention calibration for transformer-based sequential recommendation," *arXiv preprint arXiv:2308.09419*, 2023, v2, 2024.
- [10] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, 2018, pp. 197–206.
- [11] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang, "BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, 2019.
- [12] J. Li, Y. Wang, and J. McAuley, "Time interval aware self-attention for sequential recommendation," in *Proceedings of the 13th International Conference on Web Search and Data Mining (WSDM)*, 2020, pp. 322–330.
- [13] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "LightGCN: Simplifying and powering graph convolution network for recommendation," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 639–648.
- [14] J. Yu, H. Yin, X. Xia, T. Chen, L. Cui, and Q. V. H. Nguyen, "Are graph augmentations necessary? Simple graph contrastive learning for recommendation," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022, pp. 1294–1303.
- [15] J. Yu, X. Xia, T. Chen, L. Cui, N. Q. V. Hung, and H. Yin, "XSimGCL: Towards extremely simple graph contrastive learning for recommendation," *IEEE Transactions on Knowledge and Data Engineering*, 2023, arXiv:2209.02544.
- [16] Y. Hou, B. Hu, Z. Zhang, and W. X. Zhao, "CORE: Simple and effective session-based recommendation within consistent representation space," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (Short Paper)*, 2022, pp. 1796–1801.
- [17] L. Zheng, V. Noroozi, and P. S. Yu, "Joint deep modeling of users and items using reviews for recommendation," in *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining (WSDM)*, 2017.
- [18] C. Chen, M. Zhang, Y. Liu, and S. Ma, "Neural attentional rating regression with review-level explanations," in *Proceedings of the 2018 World Wide Web Conference (WWW)*, 2018, pp. 1583–1592.
- [19] M. Hu and B. Liu, "Mining and summarizing customer reviews," in *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2004, pp. 168–177.
- [20] Y. Cui, P. Zhou, H. Yu, P. Sun, H. Cao, and P. Yang, "ASKAT: Aspect sentiment knowledge graph attention network for recommendation," *Electronics*, vol. 13, no. 1, p. 216, 2024.
- [21] S. Geng, S. Liu, Z. Fu, Y. Ge, and Y. Zhang, "Recommendation as language processing (RLP): A unified pretrain, personalized prompt & predict paradigm (P5)," in *Proceedings of the Sixteenth ACM Conference on Recommender Systems (RecSys)*, 2022.
- [22] K. Bao, J. Zhang, Y. Zhang, W. Wang, F. Feng, and X. He, "TALLRec: An effective and efficient tuning framework to align large language model with recommendation," in *Proceedings of the 17th ACM Conference on Recommender Systems (RecSys)*, 2023.
- [23] A. Zhang, Y. Chen, L. Sheng, X. Wang, and T.-S. Chua, "On generative agents in recommendation," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, arXiv:2310.10108.
- [24] J. Zhang, Y. Hou, R. Xie, W. Sun, J. McAuley, W. X. Zhao, L. Lin, and J.-R. Wen, "AgentCF: Collaborative learning with autonomous language agents for recommender systems," in *Proceedings of the ACM Web Conference (WWW)*, 2024, arXiv:2310.09233.
- [25] F. Li, Q. Li, X. Gao *et al.*, "ChainRec: An agentic recommender learning to route tool chains for diverse and evolving interests," *arXiv preprint arXiv:2602.10490*, 2026.
- [26] Y. Shang, P. Liu, Y. Yan *et al.*, "AgentRecBench: Benchmarking LLM agent-based personalized recommender systems," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025.
- [27] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [28] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [29] M. Ammad-ud din, E. Ivannikova, S. A. Khan, W. Oyomno, Q. Fu, K. E. Tan, and A. Flanagan, "Federated collaborative filtering for privacy-preserving personalized recommendation system," *arXiv preprint arXiv:1901.09888*, 2019.
- [30] W. X. Zhao, Y. Hou, X. Pan, C. Yang, Z. Zhang, Z. Lin, J. Zhang, S. Bian, J. Tang, W. Sun, Y. Chen, L. Xu, G. Zhang, Z. Tian, C. Tian, S. Mu, X. Fan, X. Chen, and J.-R. Wen, "RecBole 2.0: Towards a more up-to-date recommendation library," *arXiv preprint arXiv:2206.07351*, 2022.
- [31] H. Yang, C. Zhang, and K. Li, "PyABSA: A modularized framework for reproducible aspect-based sentiment analysis," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*, 2023, pp. 5117–5122.
- [32] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [33] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018, introduces the InfoNCE loss; preprint.
- [34] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based recommendations with recurrent neural networks," in *Proceedings of the 4th International Conference on Learning Representations (ICLR)*, 2016.
- [35] L. Wu, S. Li, C.-J. Hsieh, and J. Sharpnack, "SSE-PT: Sequential recommendation via personalized transformer," in *Proceedings of the 14th ACM Conference on Recommender Systems (RecSys)*, 2020, pp. 328–337.